



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4404: Software Project Lab II

Summer, 2024–2025

Project Proposal Report

FLUX

A Visual Data Pipeline Builder

Team 5

Name	Student ID
Mahmudul Hossain Samir	230042101
Ifham Bin Hossain	230042112
Shaikh Muhaiminul Hasan	230042113

Supervisor

Samnun Azfar

Junior Lecturer, Department of CSE

Islamic University of Technology

Contents

1	Project Overview	2
2	Motivation	2
3	User Requirement Analysis	3
3.1	Real-Life Stakeholder Interview	3
3.2	Functional Requirements	4
3.3	Non-Functional Requirements	4
4	Application Flow	5
5	Key Features	6
5.1	Drag and Drop Canvas	6
5.2	Real Time Data Preview	6
5.3	Reusable Pipeline Templates	6
5.4	Pipeline to Formula Translation	6
5.5	Formula Visualizing	7
5.6	Visual Error Highlighting	7
5.7	Step by Step Replay Animation	7
5.8	Cross-Format Support	7
6	Tools and Technology	8
7	Use Cases	9
8	Proposed Timeline	10
9	Action Taken Based on Feedback	10

1 Project Overview

FLUX is a no code, drag and drop visual data pipeline builder that allows users to construct, visualize, and execute data transformation workflows entirely through a graphical canvas. Users build pipelines by connecting modular blocks each block representing a discrete operation such as filtering, lookup, or applying a formula. The system provides realtime data previews after each block, making the results of every transformation immediately visible without requiring the user to run the entire pipeline first.

At its core, FLUX aims to make spreadsheet level data processing intuitive to non technical users. Rather than requiring knowledge of complex Excel formulas or scripting languages, users can visually assemble their data logic. Crucially, FLUX supports bidirectional translation between pipelines and Excel formulas: a pipeline built on the canvas can be exported as a working Excel formula, and conversely, a pasted Excel formula can be visualized into its equivalent blocks. The application supports both `.csv` and `.xlsx` file formats throughout, and allows pipelines to be saved as reusable templates so that the same workflow can be applied to different datasets by simply swapping the source file.

2 Motivation

Data processing tasks in spreadsheet applications such as Microsoft Excel are powerful but notoriously difficult for non experts. Complex formulas become increasingly hard to read, debug, and maintain as they grow. Workflows that depend on intricate formula chains frequently break when input data changes structure, and identifying the source of such errors is a frustrating, time consuming process. Meanwhile, a large portion of data users including students, analysts, researchers, and administrators are non programmers who rely on spreadsheets for critical tasks but lack the technical background to leverage their full potential.

FLUX is motivated by the following specific problems:

- **Over Reliance on complex formulas:** Long, nested formulas are error prone and difficult to interpret even by their authors.
- **Exclusion of non technical users:** People without programming or formula expertise are effectively locked out of advanced data operations.
- **Fragile workflows:** Pipelines built with formula chains break silently when underlying data changes, with little indication of where or why.
- **Poor debugging experience:** Tracking down an error in a multistep formula requires significant manual effort.
- **Lack of transparency:** The logical steps within a formula are hidden users cannot easily see how data flows from input to output.

FLUX addresses all five of these issues by surfacing the entire data transformation process as a visual, interactive diagram.

3 User Requirement Analysis

3.1 Real-Life Stakeholder Interview

To ground the requirements in actual user needs, the team consulted a real-life stakeholder: a mid-level financial analyst at a private firm in Dhaka who regularly uses Excel for monthly budget reporting. The analyst reported spending significant time writing and debugging VLOOKUP and IF-based formulas, and expressed frustration that modifying one formula in a chain often causes other cells to produce errors that are difficult to trace. When presented with a prototype concept of FLUX, the analyst stated that a visual representation of data flow would substantially reduce time spent on debugging and would make it easier to hand reports over to colleagues unfamiliar with the formula logic.

This stakeholder interaction confirmed two high-priority requirements: (1) robust visual error highlighting that pinpoints exactly which step in the pipeline failed, and (2) seamless Excel formula export so that the resulting pipeline integrates with existing Excel-based workflows.

3.2 Functional Requirements

- Users must be able to sign up, log in, and manage their account.
- Users must be able to create a new canvas or select a saved template from the home page.
- Users must be able to upload `.csv` or `.xlsx` or both files as data sources.
- Users must be able to drag blocks from a sidebar and connect them on a canvas to form a pipeline.
- The system must display a live data preview table after each block as the pipeline is being built.
- Users must be able to paste an Excel formula and have the system reverse-engineer it into a visual pipeline.
- The system must generate the equivalent Excel formula from any pipeline built on the canvas.
- Users must be able to save a pipeline as a reusable template and reload it with a different source file.
- The system must highlight faulty blocks with a visual indicator (e.g., a red glow) and flag the affected rows.
- Users must be able to play a step by step animation of data flowing through the pipeline.
- Users must be able to export the final processed data as `.csv` or `.xlsx`.

3.3 Non-Functional Requirements

- The system must update data previews in real time with minimal perceptible latency.
- The user interface must be accessible to users with no programming background.
- The application must handle datasets of at least 50,000 rows without significant performance degradation.
- All user data and saved pipelines must be stored securely.

4 Application Flow

The high-level application flow of FLUX is illustrated in Figure 1. The system is designed around a straightforward, linear user journey that minimizes friction and keeps the focus on data transformation rather than navigation.

A user begins by signing up or logging in, after which they are directed to the Home Page, the central hub of the application. From the Home Page, the user faces a primary decision: either load an existing saved format to continue prior work, or initiate a new canvas, optionally selecting a pre-built template as a starting point. This branching structure ensures that both first-time users and returning users are served efficiently from a single entry point.

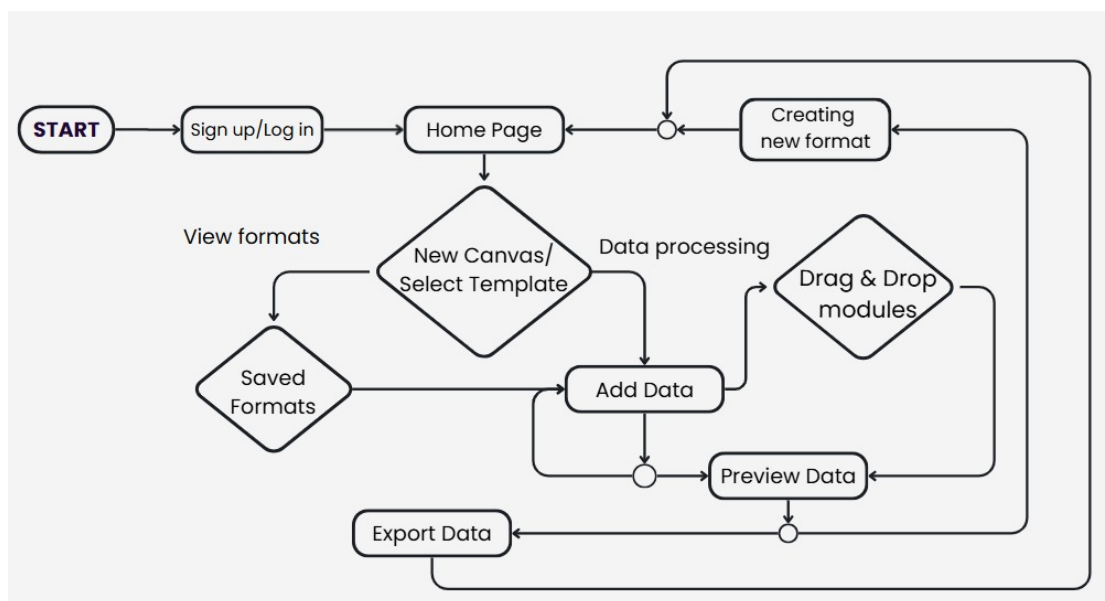


Figure 1: Application flow diagram for FLUX

Once a canvas is open, the user adds a data source and assembles transformation blocks such as filters, lookups, and formula operations using the drag-and-drop interface. FLUX renders a real-time preview after each block, allowing results to be verified incrementally. When satisfied, the user can export the processed data or save the pipeline as a reusable format, which loops back to the Home Page for future sessions. The same pipeline

5 Key Features

5.1 Drag and Drop Canvas

The primary interface of FLUX is a blank canvas on which users build their pipelines. Blocks representing individual operations (e.g., Filter, Lookup, Apply Formula, Export) are available in a sidebar and can be dragged onto the canvas and connected in sequence. No code or formula knowledge is required to assemble a pipeline.

5.2 Real Time Data Preview

After each block is placed and connected, the system immediately displays a preview of the data as it appears at that stage of the pipeline. This live feedback allows users to verify each transformation step before proceeding, dramatically reducing the chance of errors propagating silently through the pipeline.

5.3 Reusable Pipeline Templates

Completed pipelines can be saved as named templates. A saved template encapsulates the entire transformation logic without the data. To reuse it, the user simply selects the template and supplies a new source file, allowing the same workflow to be applied repeatedly without rebuilding it from scratch.

5.4 Pipeline to Formula Translation

Any pipeline assembled on the canvas can be translated into an equivalent Excel formula, generated live as the pipeline is built. The resulting formula can be copied into Excel and will produce the same output. This feature is specifically designed for users who need to deliver results in Excel but prefer to build the logic visually.

5.5 Formula Visualizing

The inverse feature allows users to paste an existing Excel formula into FLUX. The system parses the formula and renders it as a visual pipeline, showing each logical operation as a distinct block. This is particularly valuable for understanding and debugging inherited spreadsheets with opaque formula chains.

5.6 Visual Error Highlighting

When a block in the pipeline produces an error for example, due to a type mismatch or a missing column the faulty block is immediately highlighted with a red glow indicator. The specific rows affected by the error are flagged in the data preview, allowing the user to identify and correct the problem precisely.

5.7 Step by Step Replay Animation

Users can trigger an animation that plays back the data flow through the pipeline step by step. Each block lights up in sequence as data passes through it, providing a clear and intuitive understanding of how raw input is transformed into the final output.

5.8 Cross-Format Support

FLUX accepts both `.csv` and `.xlsx` files as input and can export results in either format. This allows users to work with data from a wide variety of sources without any pre-conversion.

6 Tools and Technology

Category	Technology
Backend	FastAPI(Python), Django (Python)
Frontend	React.js
Database	PostgreSQL
Version Control	Git, GitHub
IDE	Visual Studio Code
Documentation	Google Docs

- **FastAPI (Python)** – High performance async framework for efficient data transformation APIs, with automatic request validation via Pydantic.
- **Django (Python)** – Mature web framework used specifically for user authentication, registration, and the admin interface.
- **React.js** – Component based library that delivers a responsive UI with smooth drag and drop interactions.
- **PostgreSQL** – Reliable relational database for storing user accounts, saved pipelines, and template metadata.
- **Git, GitHub** – Industry standard version control and collaboration platform for tracking changes and team workflows.
- **Visual Studio Code** – Coding IDE.
- **Google Docs** - Cloud based collaborative Documentation.

7 Use Cases

FLUX is applicable across a wide range of domains:

- **Data cleaning and preprocessing:** Analysts can build filter and transformation pipelines to prepare raw datasets for downstream analysis.
- **Business reporting:** Teams can automate the preparation of dashboards and periodic reports using reusable pipeline templates.
- **Academic and research analysis:** Researchers can process survey or experimental data without requiring programming skills.
- **Financial data processing:** Accountants and analysts can model budget calculations and financial summaries visually.
- **ETL workflows:** Lightweight Extract Transform Load pipelines can be prototyped and tested without dedicated ETL infrastructure.
- **Education:** Instructors can use FLUX to teach data transformation concepts visually, making abstract operations concrete for students.
- **Spreadsheet automation:** Repetitive copy-paste or formula-application tasks can be encapsulated in templates and run in seconds.

8 Proposed Timeline

The development timeline is illustrated in Figure 2. The project spans from Week 3 to Week 14 of the semester. Core infrastructure and project setup are completed in Weeks 3–4, the formula engine is developed through Week 6, and UI development runs from Week 5 through Week 12. Advanced features are implemented between Weeks 8 and 11, user-defined module saving is addressed in Weeks 11–12, and debugging and testing run in parallel from Week 4 through Week 13.



Figure 2: Project development timeline (Weeks 3–14)

9 Action Taken Based on Feedback

The panel recommended the integration of WebSockets to support real-time collaboration, enabling multiple users to work on a shared pipeline simultaneously. While this feature is acknowledged as a valuable enhancement to the platform, its full implementation is considered a stretch goal contingent on project timeline and development capacity. After discussing with our Supervisor we have come to the conclusion that we will prioritize core functionalities first and then will introduce WebSocket-based collaboration in a later phase if time permits.